Practical Lab — Building AI Agents with LangGraph

Short Report | Advanced Artificial Intelligence / Generative AI | PhD Level

Prepared by: Arzam Yasmine

1. Introduction and Objectives

This lab explores the development of AI agents with LangChain and LangGraph. The objectives include understanding graph-based workflows, implementing conditional routing, connecting external tools, maintaining conversational state, and evaluating an Intelligent University Assistant.

2. Core Concepts

LangChain. A framework for developing AI applications by connecting language models with prompts, tools, and external data.

LangGraph. A framework for constructing graph-based AI workflows and agents. It manages shared state and controls execution across nodes.

Large Language Model (LLM). A model trained on large amounts of text to process and generate natural language for questions and tasks.

Node. A function that performs a task in a workflow. Edge. A connection that determines how execution proceeds between nodes.

START and END. The entry and termination points of a graph. State. Information shared between nodes so the workflow can preserve context and track updates.

3. Workflow and Architecture

The capstone implements an Intelligent University Assistant. A classifier routes questions to five specialized agents: Courses, Programming, Mathematics, Administration, and General. A fallback handles empty or unprocessable requests. Conditional edges direct tool calls to a shared tools node and return execution to the appropriate agent.

The graph uses a MemorySaver checkpointer and thread_id to maintain conversation history across turns. Available tools include a calculator and a current date/time tool. The architecture was exported as a separate LangGraph diagram.

4. Evaluation and Observations

Routing evaluation. The routing test classified all six questions correctly (6/6, 100% on this small test set). However, two programming classifications included extra explanatory text instead of returning only the category. Thus, semantic classification succeeded, but output formatting was not always consistent.

Suggested improvements. Structured output, few-shot examples, and validation against an allowed category enumeration would improve reliability and consistency.

Conversation memory. In the capstone test, the assistant retained the name “Ahmed” when asked “What is my name?” within the same conversation thread.

Calculator integration. The execution trace for (15 + 27) × 3 showed an AI tool call, a calculator ToolMessage returning 126, and a final assistant response reporting the result. This confirms the end-to-end tool execution path for that test.

Demonstration. The notebook also exercised course advice, Python programming, mathematics, administration, date/time questions, and empty-input handling.

5. Conclusion

The practical work demonstrates how LangGraph can coordinate specialized AI agents, conditional routing, external tools, and conversation memory within one application. The observed results validate the main workflow on the tested examples, while highlighting the importance of strict output validation and broader testing before deployment.

Supporting deliverables: Executed Jupyter notebook, README.md, source code, architecture diagram, and execution screenshots.